

TECHNICAL REFERENCE

Formula1-ML

Architecture of the prediction pipeline, the auto-updating web app, and how the two stay in sync.

Pipeline: pandas, scikit-learn · **Backend:** FastAPI, joblib · **Data:** Jolpica-F1 API · **Agent:** Claude API (tool use) · **Frontend:** vanilla JS

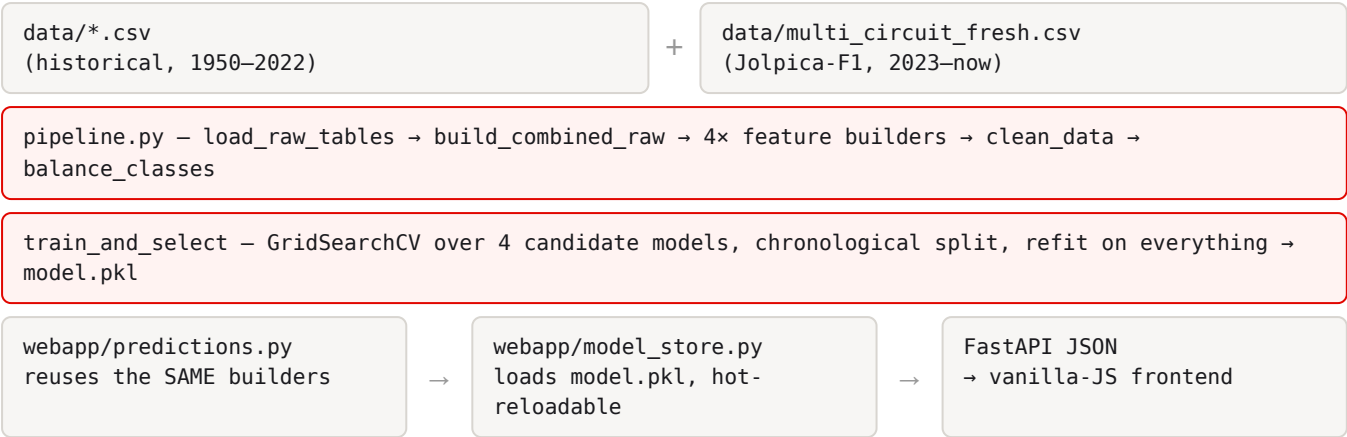
Generated as a learning reference — code excerpts are taken directly from the repository.

Contents

1.	System overview
2.	Data sources & how they combine
3.	Feature engineering — leakage-safe by construction
4.	Model selection & training
5.	The "hypothetical race" trick — predicting the unrun
6.	Web app module map
7.	Anatomy of one request
8.	The auto-update pipeline — the coordination core
9.	Crash-safety: the fingerprint sidecar
10.	Frontend structure
11.	Analytics — one EDA implementation, two surfaces
12.	Recurring design patterns
13.	File-by-file reference

1. System overview

Two halves that share one contract: `pipeline.py` 's `FEATURE_COLUMNS` / `TARGET_COLUMN` . The web app never reimplements feature logic — it imports and calls the same builder functions the training run used.



Two independent request paths inside the web app, kept deliberately separate:

PATH	ENTRY POINT	INVOLVES AN LLM?
Predictions dashboard	<code>GET /api/predictions/*</code>	No — pure feature builders → sklearn <code>.predict_proba()</code>
Race analyst	<code>POST /api/chat</code>	Yes — Claude picks tools, tools call the same PredictionService

2. Data sources & how they combine

SOURCE	COVERS	FETCHED BY	SCHEME
data/*.csv (drivers, results, races, qualifying, laptimes, pits)	1950–2022	Static, bundled with repo (Kaggle/Ergast)	Numeric driverId/raceId
data/multi_circuit_fresh.csv	2023–present, every circuit	<code>fetch_all_circuits_data.py</code> , and the auto-updater	Driver code (e.g. "VER"), synthetic raceId "2025_14"
data/singapore_fresh.csv	2023–present, Singapore only	<code>fetch_data.py</code> (legacy, pre-qualifying detail: pit stops, laps)	Driver code

Two ID schemes have to become one before any groupby/rolling-window feature can be computed across the historical/fresh boundary. `build_combined_raw()` does this once, up front:

```
# pipeline.py
def build_combined_raw(raw: dict, fresh_path: str = None) -> dict:
    # historical rows carry numeric driverId; fresh rows carry a code.
    # Reconcile onto driver CODE before concatenating -- otherwise a
    # veteran's pre-2023 rows and 2023+ rows look like two different people,
    # breaking debut-year / recent-form / circuit-history continuity.
    hist_results = raw["results"].merge(raw["drivers"][["driverId", "code"]], on="driverId")
    hist_results = hist_results.drop(columns="driverId").rename(columns={"code": "driverId"})
    ...
    return {"results": pd.concat([hist_results[cols], fresh[cols]]),
            "races": pd.concat([hist_races, fresh_races])}
```

REAL BUG THIS SHAPE CAUSED

Pre-1990s drivers have no 3-letter code (comes through as literal "\N"). A naive merge on that column collapses dozens of different people onto one join key, multiplying rows combinatorially. Fixed by filtering to `year >= MULTI_CIRCUIT_MIN_YEAR` (2008) *before* any merge — the builders still see the full timeline for debut-year lookups, but the exploding join never happens.

REAL BUG THIS SHAPE CAUSED

Pre-1990s drivers have no 3-letter code (comes through as literal "\N"). A naive merge on that column collapses dozens of different people onto one join key, multiplying rows combinatorially. Fixed by filtering to `year >= MULTI_CIRCUIT_MIN_YEAR` (2008) *before* any merge — the builders still see the full timeline for debut-year lookups, but the exploding join never happens.

Every feature answers "what did we know *before* this race started?" — enforced structurally, not by a rule someone has to remember.

Every feature answers "what did we know *before* this race started?" — enforced structurally, not by a rule someone has to remember.

FEATURE	LEAKAGE GUARD
driver_standing_before	cumsum() - points — subtracts the row's own points before ranking
driver_circuit_avg_finish	.expanding().mean().shift(1) — shift excludes the current row
driver_recent_form	.rolling(3).mean().shift(1) — same shift
constructor_recent_form	same rolling+shift, aggregated across both cars first

```
pipeline.py
```

```
def build_season_form(row: dict) -> pd.DataFrame:
    full["driver_points_before"] = (
        full.groupby(["year", "driverId"])["points"].cumsum() - full["points"]
    )
    full["driver_standing_before"] = (
        full.groupby(["year", "raceId"])["driver_points_before"].rank(method="min", ascending=False)
    )
```

```
def build_recent_form(row: dict) -> pd.DataFrame:
    full["driver_recent_form"] = (
        full.groupby("driverId")["position_num"]
        .transform(lambda s: s.rolling(RECENT_FORM_WINDOW, min_periods=1).mean().shift(1))
    )
```

Because every builder is already strictly pre-race, this same code predicts a race that *hasn't happened* — see §5.

Dropped despite being predictive

`grid_position` correlates strongly with finishing position but doesn't exist until qualifying — usually about a day before the race. Kept out of `FEATURE_COLUMNS` on purpose, trading ~4pp accuracy for being usable the moment the previous race ends rather than only the day before the next one.

```
# pipeline.py – the feature contract
NUMERIC_FEATURE_COLUMNS = ["driver_standing_before", "constructor_standing_before",
                           "years_experience", "is_rookie", "driver_circuit_avg_finish",
                           "driver_recent_form", "constructor_recent_form"]
CATEGORICAL_FEATURE_COLUMNS = ["circuit_id"]
TARGET_COLUMN = "points_finish" # position 1..POINTS_POSITIONS
POINTS_POSITIONS = 10 # F1 has paid points to P10 since 2010
```

4. Model selection & training

- 1 **Clean** — drop rows missing any `REQUIRED_COLUMNS` or the target.
- 2 **Chronological split** — most recent seasons held out as test, *not* a random shuffle. A random split would let the model be judged on data chronologically before some of its own training rows.
- 3 **Balance** — upsample the minority class, *after* the split, on the training fold only (upsampling before the split risks a duplicated row landing in both train and test).
- 4 **Compare 4 candidates** — Random Forest, Logistic Regression, SVM, HistGradientBoosting — each baseline vs. `GridSearchCV`-tuned.
- 5 **Refit winner on everything** — train+test years combined, since the shipped model should use every real result available.

pipeline.py — chronological_split

```
def chronological_split(df, test_size=0.25, test_circuit_id=None):
    eval_scope = df if test_circuit_id is None else df[df["circuit_id"] == str(test_circuit_id)]
    # walk backward from the newest year, accumulating rows until ~25% held out
    test_years, running_total = [], 0
    for year in reversed(year_counts.index.tolist()):
        if running_total >= n_test_target: break
        test_years.append(year); running_total += year_counts[year]
```

Two training runs happen every time `pipeline.py` executes, and whichever generalizes better gets exported:

RUN	TRAINED ON	EVALUATED ON
Singapore-only	Singapore GP, every year	held-out recent Singapore years
Multi-circuit	every circuit, 2008+	held-out years, Singapore rows only (<code>test_circuit_id</code>) — tests "does training everywhere help predict here"

Multi-circuit has won every run so far (0.820 held-out accuracy on the current points-finish target, HistGradientBoostingClassifier) and is what's actually deployed.

5. The "hypothetical race" trick

How a model trained only on *results* predicts a race with no result yet.

Because every feature builder is already strictly pre-race (§3), a placeholder row — real driver, real constructor, real circuit, `position = "\N"` — run through the same builders yields exactly the features that race would have had. Nothing from the future leaks in, because there is no future in the row.

webapp/predictions.py

```
def upcoming_features(self, circuit_id, entrants, year, round_):
    placeholder_results = pd.DataFrame([
        {"raceId": race_id, "driverId": code, "constructorId": constructor,
         "position": NO_RESULT, "points": 0}
        for code, constructor in entrants
    ])
    scoped = {"results": pd.concat([self.combined["results"][cols], placeholder_results]),
              "races": pd.concat([self.combined["races"], placeholder_race])}
    return self._apply_builders(scoped) # same 4 builders as training, unmodified
```

Where "next race" always slots in: **immediately after the last completed race**, regardless of which circuit is being asked about.

REAL BUG THIS CAUGHT

Slotting a placeholder into a season with zero completed races ties every driver's `driver_standing_before` at P1 (everyone's cumulative points are 0) — silently destroying the model's strongest feature and leaving it to guess from circuit history alone. Fixed: features are always computed at `next_round_slot()` (the real "day after the last race" position), and only the *label* shown to the user reflects which future round was actually asked about.

6. Web app module map

MODULE	OWNS
<code>main.py</code>	FastAPI app, route wiring, module-level singletons (service/model_store/analyst/live/news/auto_update)
<code>model_store.py</code>	<code>ModelStore</code> — loads model.pkl, exposes <code>.reload()</code> for hot-swap without a restart
<code>predictions.py</code>	<code>PredictionService</code> — feature building, the placeholder-row trick, rank-based top-N labeling
<code>reference.py</code>	<code>Reference</code> — driver/team/circuit display names, colors; era-collision handling for reused driver codes
<code>live_data.py</code>	<code>LiveData</code> — Jolpica fetch + on-disk cache; <code>refresh_season()</code> , <code>season_roster()</code> , <code>schedule()</code>
<code>scheduler.py</code>	<code>AutoUpdateScheduler</code> — background thread: check → fetch → publish → retrain → swap (§8)
<code>agent.py</code>	<code>RaceAnalyst</code> — Claude tool-use loop over read-only wrappers around PredictionService
<code>news.py</code>	<code>NewsService</code> — RSS fetch, keyword-based topic tagging, long TTL cache
<code>static/app.js</code>	All frontend state + rendering, no framework
<code>../analytics.py</code>	Season EDA over the model's own features — imported by <code>main.py</code> and by the notebook (§11)

Construction order in `main.py` matters — each singleton depends on the previous:

```
reference      = Reference(pipeline.load_raw_tables(DATA_DIR))
model_store   = ModelStore()
service       = PredictionService(DATA_DIR, model_store, reference)
analyst       = RaceAnalyst(service)
live          = LiveData(DATA_DIR)
auto_update   = AutoUpdateScheduler(live, DATA_DIR, model_store, _rebuild_service, PROJECT_ROOT)
auto_update.start() # daemon thread begins here, at import time
```

7. Anatomy of one request

GET `/api/predictions/upcoming?circuit=monza`, end to end:

- 1 `main.py` route handler calls `service.latest_entry_list()` — the 20 (driver, constructor) pairs from the most recent race.
- 2 `service.next_round_slot()` — reads `race_catalog()`, returns `(year, last_round + 1)`.
- 3 `service.predict_upcoming(circuit, entrants, year, round_)` — memoized via `@functools.lru_cache`, keyed on all four args.
- 4 Cache miss → `upcoming_features()` builds the placeholder row, runs the 4 feature builders (\$5).
- 5 `model_store.predict_points_proba(features)` — `model.predict_proba(features)[: , class_1_index]`.
- 6 `_to_rows()` sorts by probability descending, marks `predicted_points = position < 10` (rank-based, not a 50% cutoff — see §12).
- 7 Response JSON assembled with `reference.driver()` / `.team()` for display names.

8. The auto-update pipeline — the coordination core

A daemon thread, started once at import time, living inside the same `uvicorn` process — not a cron job, not a separate service.

`webapp/scheduler.py`

```
def _loop(self):
    while not self._stop.is_set():
        self.check_now()
        self._stop.wait(CHECK_INTERVAL_SECONDS)  # default 3 hours
```

- 1 **Check** — `live.refresh_season(year)` fetches that season from Jolpica, diffs row count against `data/multi_circuit_fresh.csv`.
- 2 **Publish immediately** — if changed, `rebuild_service()` runs *before* retraining. Predictions reflect the new race's standings/form within seconds; the model's learned weights lag until step 4.
- 3 **Retrain as a subprocess** — `subprocess.run([sys.executable, "pipeline.py", ...])`, not an in-process call: a ~20-30 min GridSearchCV sweep would otherwise stall the event loop, and a crash there can't take the API down.
- 4 **Atomic swap** — `os.replace(tmp_path, model.pkl)`, then `model_store.reload()`. Every component already holds a reference to the same `ModelStore` object, so overwriting `.model` inside it is visible everywhere instantly — no restart.

```
def check_now(self, force=False):
    if not self._busy.acquire(blocking=False):
        return {"ok": False, "busy": True}  # scheduled + manual can't race each other
    ...
    if result.get("changed"):
        self.rebuild_service()  # step 2 -- fast
        self.last_data_refresh = time.time()
        self._retrain()  # step 3+4 -- slow
```

Why a subprocess, concretely

```
os.replace(tmp_path, self.model_store.path)  # atomic on same filesystem --
                                              # a request mid-swap sees either the
                                              # whole old file or the whole new one,
                                              # never a partial write

self.model_store.reload()
self.rebuild_service()  # feature tables were built pre-retrain, redo them
```

9. Crash-safety: the fingerprint sidecar

"Did a fetch change anything?" isn't enough on its own — it can't recover from an interrupted retrain.

THE FAILURE MODE THIS FIXES

Server restarts mid-retrain (deploy, laptop sleep) → new race data already on disk, model never replaced. Next boot asks "did the last fetch add rows?" — no, they're already there — and never retrains again, silently staying one race behind forever.

webapp/scheduler.py

```
def _data_fingerprint(self) -> str:
    frame = pd.read_csv(fresh_csv_path, usecols=["year", "round"])
    latest = frame.sort_values(["year", "round"]).iloc[-1]
    return f"{latest.year}_{latest.round}_{len(frame)}"

def model_is_stale(self) -> bool:
    return self._data_fingerprint() != self._trained_fingerprint() # reads model.pkl.meta.json
```

Written only *after* the swap succeeds — a fingerprint recorded before the model actually landed would mark an interrupted retrain as done:

```
fingerprint_at_start = self._data_fingerprint()
... subprocess.run(pipeline.py) ...
os.replace(tmp_path, self.model_store.path)
self._record_trained_fingerprint(fingerprint_at_start) # only now
```

`model.pkl.meta.json` is gitignored — it describes one machine's local model, and committing it would tell a fresh clone the checked-in model.pkl is current when it almost certainly isn't.

10. Frontend structure

No framework — one `state` object, one `el()` helper, direct DOM construction.

```
const el = (tag, className, text) => {
  const node = document.createElement(tag);
  if (className) node.className = className;
  if (text !== undefined) node.textContent = text; // never innerHTML for data --
  return node; // XSS-safe by construction
};
```

Four independent render paths, mirroring the backend split (§1). None calls another; each owns its own `state.*Loaded` flag:

ENTRY POINT	ENDPOINT	LOADED
<code>loadPredictions()</code>	<code>/api/predictions/*</code>	At boot
<code>loadAnalytics()</code>	<code>/api/analytics</code>	On first tab show
<code>loadNews()</code>	<code>/api/news</code>	On first tab show
<code>sendChat()</code>	<code>/api/chat</code>	On submit

The two lazy tabs are wired in one line each inside `showTab()` — no router, no lifecycle framework:


```
function showTab(name) {
  $$(".tab").forEach((tab) => tab.classList.toggle("is-active", tab.id === `tab-${name}`));
  if (name === "news"      && !state.newsLoaded)      loadNews();
  if (name === "analytics" && !state.analyticsLoaded) loadAnalytics();
}
```

Era-collision gating (photos)

Driver codes repeat across eras (**VER** = Verstappen today, Vergne 2012–14). A photo keyed only by code would risk showing the wrong face on an old replay:

```
const isMostRecentReplay =
  isReplay && state.races.length > 0 && payload.race.race_id === state.races[0].race_id;
const showDriverPhoto = !isReplay || isMostRecentReplay; // team badges have no such collision
```

11. Analytics — one EDA implementation, two surfaces

The Analytics tab and the project notebook show the same numbers because they call the same function.

`analytics.py` is the only implementation; neither surface computes anything itself.

THE FAILURE MODE THIS AVOIDS

Two "official" answers to the same question. A page that queries one way and a notebook that recomputes another drift the moment either changes — and nothing tells you which one is wrong.

`analytics.py` — the split that makes sharing possible

```
def build_season_frame(data_dir="data", year=None):
    # Standalone: rebuilds the frame from the CSVs via pipeline.py's own
    # builders. The notebook has no PredictionService, so it needs this.
    ...
    return base[base["year"] == year].copy(), year

def compute(frame: pd.DataFrame, year: int) -> dict:
    # PURE -- takes an already-built frame. That one design choice is what
    # lets the web app hand over its cached table instead of rebuilding
    # the whole timeline on every request.
```

CALLER	WHERE THE FRAME COMES FROM	COST PER CALL
<code>GET /api/analytics</code>	<code>service.known_features()</code> — already cached at startup	Aggregation only
Notebook cell	<code>build_season_frame("data")</code>	Full rebuild (once)

`webapp/main.py`

```
@app.get("/api/analytics")
def get_analytics(year: int = None):
    features = service.known_features()
    seasons = sorted({int(v) for v in features["year"].dropna().unique()}, reverse=True)
    target = int(year) if year is not None else seasons[0]
    report = analytics.compute(features[features["year"] == target], target)
    # Ids are what the frame carries; only the UI wants display names.
    for row in report["by_team"]:
        row["name"] = reference.team(row["team"])["name"]
    return report
```

Why one season, not all of them

`driver_standing_before` and the rolling-form features are *within-season* quantities — a championship position only means something relative to the season it was computed in. Pooling 2008–2025 would average across grids that share no drivers, cars or regulations, and a reader consulting analytics to reason about the next race wants this year's competitive order.

Two definitional decisions worth copying

```
def _scored(frame):
    # NaN, not False, for a DNF -- excluded from every rate rather than
    # silently counted as a failure, which conflates "wasn't fast enough"
    # with "didn't finish".
    return frame["actual_position"].between(1, pipeline.POINTS_POSITIONS)

# Quantile buckets, not equal-width: championship standing is uniform but
# circuit-average-finish clusters hard, and equal-width bins there produce
# near-empty extremes whose rates swing on two or three drivers.
binned = pd.qcut(usable[feature], q=buckets, duplicates="drop")
```

Charts: hand-rolled SVG, real pixels

No charting library — same no-build-step stance as the rest of the frontend. Charts are drawn in pixel coordinates measured off their container rather than in a `viewBox` scaled to fit, because a stretched `viewBox` shrinks the tick labels along with the bars.

REAL BUG THIS SHAPE CAUSED

An `auto-fit` grid gives its first child the whole row until siblings arrive. Measuring each small multiple as it was appended read a width that cell would not keep — so `preserveAspectRatio` letterboxed chart #1 down to a third of its intended height, labels and all. Fixed by building every cell first, then measuring:

```
// pass 1: build and append, measuring nothing
const cells = distributions.filter((e) => e.counts.length).map((entry) => {
    const plot = el("div"); cell.appendChild(plot); container.appendChild(cell);
    return { entry, plot };
});
// pass 2: the grid has settled, so clientWidth is now the real column
cells.forEach(({ entry, plot }) => { const width = chartWidth(plot, 240); ... });
```

Colour follows the job, not the entity. One hue for every single-series chart (categories are already named on the axis, so a second hue would encode nothing); a two-hue diverging scale *only* on the correlation chart, where the sign genuinely is the message. The pair was validated against this app's dark surface rather than assumed — CVD ΔE 19.2, normal-vision ΔE 29.0, both clear of the 8 / 15 floors.

The notebook side

The notebook stays what it has always been — data cleaning and EDA, never training. Its final section is three lines of setup and then plots of a dictionary it did not compute:

SC1015 Formula 1 Mini-Project.ipynb – final section

```
import analytics

season_frame, season_year = analytics.build_season_frame("data")
report = analytics.compute(season_frame, season_year)

# ... every cell below reads report[...] -- nothing is recomputed here
correlations = pd.DataFrame(report["correlations"])
plt.barh(order["label"], order["correlation"], color=colours)
```

So a fix to a definition — how a DNF is treated, where a bucket boundary falls — lands in one file and both surfaces move together.

12. Recurring design patterns

Rank-based, not threshold-based, classification

```
// WRONG -- disagrees with what the sorted-by-probability UI visibly shows
predicted_points = probability >= 0.5

// RIGHT -- a points finish always names exactly POINTS_POSITIONS drivers
predicted_points = position_in_sorted_list < POINTS_POSITIONS
```

The threshold version let a borderline driver at rank 10 with 42% probability read as "not predicted" while still visibly sitting in the top-10 rows — a real miss then displayed as "called right" when the driver finished outside the points and the (wrong) prediction also said "no."

Cache-poisoning-proof caching

```
# webapp/images.py's predecessor (removed, see §13) -- the general shape
if not response.ok:
    return miss          # NEVER cached -- a 403 today isn't "no photo forever"
result = {...}
self._cache[title] = result  # only a genuine answer gets remembered
```

Paired with a `CACHE_VERSION` constant bumped whenever a bug could have written bad entries — old-version cache files are discarded wholesale on load rather than trusted.

Classify by what the reader wants, not by what the source is

The news tab filters by topic, not by outlet — "show me driver news" is a question a reader has; "show me one site's coverage" is not. Tagging happens server-side on title + summary, and a headline can carry more than one tag:

webapp/news.py

```
CATEGORIES = ["Drivers", "Teams", "Regulations", "Race Weekend", "Other"]

# Keyword sets are seeded from the data the app already has -- current
# grid surnames and team names -- so a new driver is matchable the moment
# the roster refresh picks them up, with no keyword list to maintain.
DRIVER_KEYWORDS = {surname.lower() for _, surname, _ in CURRENT_GRID.values()} | {
    "driver", "rookie", "seat", "signing", "contract", "debut", ...}

def _categorize(title: str, summary: str) -> list:
    text = f"{title} {summary}".lower()
    hits = []
    if any(k in text for k in DRIVER_KEYWORDS): hits.append("Drivers")
    ...
    return hits or ["Other"]  # never silently drop a headline
```

Graceful degradation, everywhere external

DEPENDENCY	IF UNREACHABLE
Jolpica-F1 API	Falls back to bundled CSVs; <code>live.online</code> reported in <code>/api/status</code>
ANTHROPIC_API_KEY unset	Assistant tab reports unavailable; dashboard unaffected
RSS feed down	Other feeds still return; error message names which one and why
Local photo missing	404 → frontend falls back to colored-initials monogram

13. File-by-file reference

FILE	ROLE
<code>pipeline.py</code>	Single source of truth: features, target, training, export. Everything else imports from here.
<code>fetch_all_circuits_data.py</code>	Pulls one season, every circuit, from Jolpica — feeds <code>multi_circuit_fresh.csv</code>
<code>fetch_data.py</code>	Legacy: Singapore-only detail fetch (laps, pit stops) — imports <code>TARGET_COLUMN</code> / <code>POINTS_POSITIONS</code> from <code>pipeline.py</code> rather than hardcoding them, after a drift bug silently dropped every fresh row it produced
<code>analytics.py</code>	§11 — season EDA over the model's own features; the one implementation behind both the Analytics tab and the notebook
<code>SC1015ipynb</code>	Data cleaning + EDA only — never trains. Its last section imports <code>analytics.py</code> rather than recomputing anything.
<code>f1_agent.py</code>	Standalone CLI tool-use demo, separate from the web app's <code>agent.py</code> , sharing the same <code>model.pkl</code>
<code>webapp/main.py</code>	FastAPI routes + singleton wiring
<code>webapp/scheduler.py</code>	§8 — the coordination core
<code>webapp/predictions.py</code>	§5 — feature reuse, placeholder-row trick, rank-based labeling
<code>webapp/model_store.py</code>	Loads/reloads <code>model.pkl</code> ; never trains
<code>webapp/live_data.py</code>	Jolpica fetch + disk cache; season roster; calendar
<code>webapp/reference.py</code>	Display names/colors; era-collision handling
<code>webapp/agent.py</code>	Claude tool-use loop, read-only tools over <code>PredictionService</code>
<code>webapp/news.py</code>	RSS fetch, topic tagging, long-TTL cache
<code>webapp/static/app.js</code>	All frontend logic, no build step — including §11's hand-rolled SVG charts
<code>webapp/static/photos/</code>	Locally-bundled driver/team images — see <code>photos/README.md</code>
<code>model.pkl</code>	Committed, trained artifact — kept current by §8/§9
<code>model.pkl.meta.json</code>	Gitignored — §9's fingerprint, machine-local